K8S-10: Metadata Telemetry Enrichment

> **Series:** K8S — Kubernetes Monitoring | **Notebook:** 10 of 13 | **Created:** January 2026 | **Last Updated:** 07/15/2026

Enriching All Telemetry with Kubernetes Metadata

Kubernetes metadata enrichment automatically adds labels and annotations from your Kubernetes resources to all telemetry signals. This is the **recommended approach** for adding context to your observability data because it enriches everything: metrics, logs, traces, events, and entities.

Table of Contents

OneAgent Attribute Enrichment (OneAgent 1.331+)

> **Requires:** OneAgent version **1.331** or later

OneAgent can enrich **all telemetry signals** (metrics, spans, logs, events, entities) with custom metadata at the source — before data reaches the Dynatrace platform. This is more efficient than server-side tagging (auto-tags) because enrichment happens on the host and propagates to all Smartscape nodes.

Primary Fields (standardized from Semantic Dictionary):
- `dt.security_context` — data governance and access control
- `dt.cost.costcenter` — cost allocation
- `dt.cost.product` — product attribution

Primary Tags (custom key-value pairs):
- `primary_tags.environment` — environment identification (production, staging, etc.)

- `primary_tags.team` — team ownership
- `primary_tags.business_unit` — organizational unit

Configuration:

```
# Set during OneAgent installation
Dynatrace-OneAgent-Linux.sh --set-host-tag="primary_tags.environment=production" --set-host-
tag="dt.security_context=confidential"

# Set on existing agents via oneagentctl
oneagentctl --set-host-tag="primary_tags.environment=production"
oneagentctl --set-host-tag="dt.cost.costcenter=12345"

# Per-process via environment variable (overrides host-level)
DT_TAGS="primary_tags.team=platform primary_tags.environment=production"
```

Benefits over auto-tagging:
Aspect	Auto-Tags (Server-Side)	Attribute Enrichment (Agent-Side)
When applied	After data arrives at platform	At the source, before transmission
Scope	Entity tags only	All signals: metrics, spans, logs, events, entities
Grail integration	Limited	Full — feeds OpenPipeline routing, bucket assignment, permissions
Cost allocation	Not supported	`dt.cost.costcenter`, `dt.cost.product` fields
Security context	Not supported	`dt.security_context` for data governance

> **See:** Primary Grail fields and tags enrichment through OneAgent

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail
Permissions	`settings.read`, `settings.write`
DynaKube	Deployed with `metadataEnrichment` enabled
Kubernetes	Namespaces with labels/annotations to enrich

1. Why Metadata Enrichment?

Kubernetes metadata enrichment solves a critical observability challenge: **connecting telemetry to business context**.

Common Use Cases

Use Case	Metadata Example	Benefit
Cost allocation	`cost-center: finance`	Attribute costs to teams

Use Case	Metadata Example	Benefit
Environment identification	`env: production`	Filter by deployment stage
Team ownership	`team: checkout`	Route alerts to owners
Compliance tagging	`compliance: pci-dss`	Identify regulated workloads
Application grouping	`app: ecommerce`	Group related services

What Gets Enriched

With settings-based enrichment, **all signals** receive metadata:

Signal Type	Enriched?	Notes
Logs	Yes	All container and pod logs
Metrics	Yes	Including Kubernetes platform metrics
Spans/Traces	Yes	Distributed tracing data
Events	Yes	Kubernetes events
Entities	Yes	Smartscape topology

Primary Grail Fields and Tags

In addition to settings-based enrichment, Dynatrace automatically propagates certain **Primary Grail Fields** across all signal types. These fields are indexed, require no configuration, and are the foundation for cross-signal data organization:

Primary Grail Field	DQL Field	Propagation
Kubernetes Cluster	`k8s.cluster.name`	All signals
Kubernetes Namespace	`k8s.namespace.name`	All signals
Host Group	`dt.host_group.id`	All signals
AWS Account	`aws.account.id`	All signals

Primary Grail Tags (prefix `primary_tags.`) extend this concept for custom dimensions like `primary_tags.app` or `primary_tags.team`. Use them when Primary Grail Fields don't align with your organizational structure (e.g., shared infrastructure hosting multiple applications).

Setting Primary Tags Directly from Kubernetes (June 2026)

The Kubernetes tag setup (DT docs) page documents dedicated annotations for emitting primary tags at namespace or pod scope:

```
metadata:
  annotations:
    metadata.dynatrace.com/primary_tags.team: payments
    metadata.dynatrace.com/primary_tags.environment: production
```

Primary fields ride the same surface (`metadata.dynatrace.com/dt.security_context` , `metadata.dynatrace.com/dt.cost.costcenter` , `metadata.dynatrace.com/dt.cost.product`). Precedence is documented as pod over namespace: *"When the same key is set at both scopes, the pod-level value wins for that pod."* With Dynatrace Operator 1.10.0 the documented specificity chain extends to four levels (highest wins): pod annotations → namespace annotations → DynaKube resource attributes (see §3) → central configuration rules.

Two adoption caveats:

- **Version gates (corrected 07/31/2026).** These notebooks previously carried the quoted sentence *"the at-source and central configuration options require minimum component versions: OneAgent version 1.343+, ActiveGate version 1.341+, Dynatrace Operator version 1.10+"*. That no longer matches the source pages, and it conflated two capabilities with different floors. The current published figures:

Capability	Minimum version	Source
At-source / static tag enrichment (host tags, `DT_TAGS`)	**OneAgent 1.333+**	OneAgent tag setup (DT docs), OneAgent attribute enrichment (DT docs)
Central enrichment configuration	**OneAgent 1.343**	What's new in OneAgent 1.343 (DT docs) — *"OneAgent now enriches telemetry data at the source based on a central enrichment configuration defined in the platform"*
Kubernetes telemetry enrichment	**Operator 1.10+, OneAgent 1.333+, ActiveGate 1.343+**	Kubernetes tag setup (DT docs)

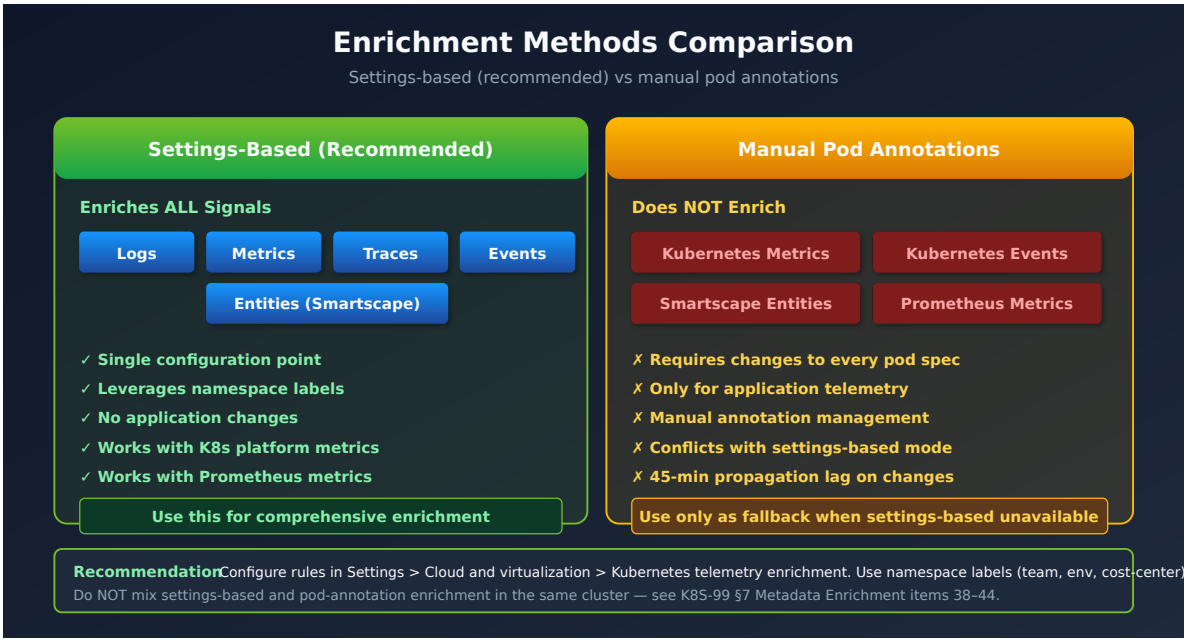
Two practical consequences. **At-source tagging is available far earlier than previously stated** — a 1.333–1.342 fleet can already emit primary tags at source and only lacks the *central* configuration surface. And the ActiveGate figure was **1.343+**, not 1.341+. OneAgent 1.343 has shipped. As always, tenant version is not agent version: verify your own fleet before relying on either floor.
- **Central configuration ships with SaaS 1.343 (July 2026 — staged tenant rollout):** rules that promote existing namespace labels to primary tags without manifest changes (limited to 20 rules per scope) arrive as *Centralized telemetry metadata enrichment* — key-value pairs, namespace annotations, and domain tags managed centrally. The docs previously flagged this as *"the recommended approach"* arriving mid-2026; it is the documented forward path alongside the settings-based enrichment described in this notebook — which remains the working path until 1.343 reaches your tenant (verify availability before designing against the central rules). On the ActiveGate side, AG 1.341 adds full namespace- and pod-level `metadata.dynatrace.com/primary_tags.<key>` enrichment and primary-Grail-field enrichment (`dt.security_context` , `dt.cost.product`) for Prometheus metrics monitored via ActiveGate. Note the remaining version gate: the at-source annotation path documents OneAgent 1.343+ (not yet released as of July 2026), ActiveGate 1.341+ (shipped), Operator 1.10+ (shipped July 15, 2026).

Cost allocation fields like `dt.cost.costcenter` and `dt.cost.product` also propagate to service metrics, enabling chargeback reporting across teams.

> **See also: ORGNZ-10: Advanced Segment Definitions** covers Primary Grail Fields in depth, including enrichment approaches for dedicated vs. shared infrastructure scenarios.

2. Enrichment Methods Comparison

Dynatrace offers multiple ways to add Kubernetes metadata. Choose based on your needs:



Why Settings-Based is Recommended

Settings-based enrichment:
- Enriches **all signals**: logs, metrics, traces, events, entities
- Single configuration point (Dynatrace Settings)
- Leverages existing namespace labels
- No application changes required
- Works with Kubernetes platform metrics

Manual pod annotations:
- Does NOT enrich: Kubernetes metrics, events, Smartscape entities, Prometheus metrics
- Requires changes to every pod spec

> **Warning:** Do not mix settings-based enrichment with manual pod annotations. Using both simultaneously may cause conflicts and unexpected behavior.

3. DynaKube Configuration

Metadata enrichment must be enabled in your DynaKube custom resource.

Enable Metadata Enrichment

```
apiVersion: dynatrace.com/v1beta5
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  apiUrl: https://ENVIRONMENT_ID.live.dynatrace.com/api

  # Enable metadata enrichment (enabled by default)
  metadataEnrichment:
    enabled: true

  oneAgent:
    cloudNativeFullStack: {}

  activeGate:
    capabilities:
      - kubernetes-monitoring
```

Disable Metadata Enrichment (if needed)

```
# Using annotation
kubectl annotate dynakube -n dynatrace dynakube \
  feature.dynatrace.com/disable-metadata-enrichment="true"
```

Or in the DynaKube spec:

```
spec:
  metadataEnrichment:
    enabled: false
```

Defining Resource Attributes in the DynaKube (Operator 1.10.0+)

Dynatrace Operator **1.10.0** (released July 15, 2026) adds a cluster-scoped enrichment surface: static resource attributes defined directly in the DynaKube spec. The Kubernetes tag setup (DT docs) page positions this mechanism in the primary-tag specificity chain below pod/namespace annotations and above central configuration rules. Three spec fields participate:

Spec field	Applies to	Precedence
`.spec.resourceAttributes`	All signals (OneAgent, OTLP, log monitoring, ActiveGate)	Base — overridden by the mode-specific fields below on duplicate keys
`.spec.oneAgent.<mode>.additionalResourceAttributes`	OneAgent-emitted signals only	Wins over `.spec.resourceAttributes` on duplicate keys

Spec field	Applies to	Precedence
`.spec.otlpExporterConfiguration.additionalResourceAttributes`	OTLP telemetry only	Wins over `.spec.resourceAttributes` on duplicate keys

```
spec:
  resourceAttributes:
    aws.account.id: "123456789012"
  oneAgent:
    cloudNativeFullStack:
      additionalResourceAttributes:
        my.team: platform
  otlpExporterConfiguration:
    additionalResourceAttributes:
      my.team: platform
```

Constraints documented with the feature:

- A **soft limit of 10 attributes** applies across `.spec.resourceAttributes` and all `additionalResourceAttributes` blocks combined.
- Keys are sanitized (invalid DNS characters replaced); keys over 63 characters violate Kubernetes label limits.
- When both OneAgent and OTLP injection are active on the same pod, **conflicting keys produce undefined behavior** — keep the two mode-specific blocks consistent.
- The values are cluster-scoped and static. Use pod/namespace annotations for per-workload values; use this surface for cluster-wide facts (account IDs, environment, owning platform team).

> **Running an earlier Operator?** These spec fields require Operator 1.10.0+ — on earlier versions the DynaKube rejects them at admission. The namespace/pod annotation path and the settings-based enrichment described in this notebook remain the working paths until your clusters upgrade.
>
> **See:** Metadata enrichment (DT docs), Operator 1.10.0 release notes (DT docs)

4. Configuring Settings-Based Enrichment

Access Settings

1. Navigate to **Settings** > **Cloud and virtualization** > **Kubernetes telemetry enrichment**
2. Select **Add rule**

Rule Configuration Options

Field	Description	Example
Metadata type	Source type (annotation or label)	`Label`
Key	The label/annotation key to capture	`team`
Prefix	Optional prefix for the enriched attribute	`k8s.`

Example: Capture Team Label

If your namespaces have:

```
apiVersion: v1
kind: Namespace
metadata:
  name: checkout
  labels:
    team: checkout-team
    cost-center: engineering
    env: production
```

Create enrichment rules:

Rule	Metadata Type	Key	Prefix
1	Label	`team`	`k8s.`
2	Label	`cost-center`	`k8s.`
3	Label	`env`	`k8s.`

Result: All telemetry from the `checkout` namespace will have:
- `k8s.team: checkout-team`
- `k8s.cost-center: engineering`
- `k8s.env: production`

5. Namespace Selectors

By default, enrichment rules apply to **all namespaces**. Use namespace selectors to limit scope.

DynaKube Namespace Selector

If your DynaKube uses a `namespaceSelector`, ensure it matches the namespaces you want to enrich:

```
spec:
  metadataEnrichment:
    enabled: true
    namespaceSelector:
      matchLabels:
        dynatrace-enrich: enabled
```

Labeling Namespaces for Enrichment

```
# Enable enrichment for a namespace
kubectl label namespace checkout dynatrace-enrich=enabled

# Verify labels
kubectl get namespace checkout --show-labels
```

Match Expressions

For more complex selection:

```
namespaceSelector:
  matchExpressions:
    - key: env
      operator: In
      values:
        - production
        - staging
```

6. Understanding Enrichment Files

The Dynatrace Operator creates enrichment files that are used by OneAgent and applications.

File Locations

File	Location	Purpose
`dt_metadata.json`	`/var/lib/dynatrace/enrichment/`	JSON format metadata
`dt_metadata.properties`	`/var/lib/dynatrace/enrichment/`	Properties format
`dt_host_metadata.json`	`/var/lib/dynatrace/enrichment/`	Host-level metadata

Example dt_metadata.json Content

```
{
  "k8s.namespace.name": "checkout",
  "k8s.pod.name": "checkout-api-7d9f8c6b4d-x2k9m",
  "k8s.team": "checkout-team",
  "k8s.cost-center": "engineering",
  "k8s.env": "production"
}
```

Accessing Enrichment Files in Applications

For manual instrumentation or custom applications:

```
import json
import os

# Load Dynatrace metadata
enrichment_paths = [
    '/var/lib/dynatrace/enrichment/dt_metadata.json',
    '/var/lib/dynatrace/enrichment/dt_host_metadata.json'
]

metadata = {}
for path in enrichment_paths:
    if os.path.exists(path):
        with open(path) as f:
            metadata.update(json.load(f))

print(metadata)
```

7. Querying Enriched Data

Once enrichment is configured, you can filter and group by the enriched attributes.

```
// Query logs by enriched team label
fetch logs, from:-1h
| filter isNotNull(k8s.namespace.name)
| summarize logCount = count(), by:{k8s.namespace.name}
| sort logCount desc
| limit 20
```

```
// Group metrics by cost center (enriched label)
timeseries avgCpuMillicores = avg(dt.kubernetes.container.cpu_usage), from:-1h, by:
{k8s.namespace.name}
| sort avgCpuMillicores desc
```

```
// Find all spans from production environment
fetch spans, from:-1h
| filter isNotNull(k8s.namespace.name)
| summarize
    spanCount = count(),
    avgDuration = avg(duration),
    by:{k8s.namespace.name, service.name}
| sort spanCount desc
| limit 20
```

8. Use Cases

Cost Allocation by Team

Label namespaces with cost centers:

```
kubectl label namespace checkout cost-center=checkout-team
kubectl label namespace catalog cost-center=catalog-team
kubectl label namespace shared cost-center=platform
```

Create enrichment rule for `cost-center` label, then query:

```
timeseries totalCpu = sum(dt.kubernetes.container.cpu_usage), from:-1h, by:{k8s.cost-center}
```

Pipeline Routing

Route logs to different buckets based on enriched metadata. In OpenPipeline:

```
processors:
  - type: route
    rules:
      - condition: "k8s.env == 'production'"
        destination: "prod_logs_365d"
      - condition: "k8s.env == 'staging'"
        destination: "staging_logs_35d"
```

Security Context Assignment

Use enriched metadata in security context for fine-grained access:

```
processors:
  - type: security-context
    rules:
      - condition: "k8s.team == 'checkout-team'"
        context: "team:checkout"
```

Grail Permissions

Create IAM policies based on Kubernetes attributes:

```
ALLOW storage:buckets:read WHERE storage:bucket-name STARTSWITH "default_";
ALLOW storage:logs:read WHERE storage:k8s.namespace.name = "checkout";
```

9. Settings API

Manage enrichment rules programmatically via the Settings API.

Schema

The schema for Kubernetes telemetry enrichment is: `builtin:kubernetes.generic.metadata.enrichment`

List Current Rules

```
curl -X GET "https://ENVIRONMENT_ID.live.dynatrace.com/api/v2/settings/objects" \
  -H "Authorization: Api-Token YOUR_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "schemaIds": ["builtin:kubernetes.generic.metadata.enrichment"],
    "scopes": ["environment"]
  }'
```

Create Rule via API

```
curl -X POST "https://ENVIRONMENT_ID.live.dynatrace.com/api/v2/settings/objects" \
  -H "Authorization: Api-Token YOUR_TOKEN" \
  -H "Content-Type: application/json" \
  -d '[
    {
      "schemaId": "builtin:kubernetes.generic.metadata.enrichment",
      "scope": "environment",
      "value": {
        "enabled": true,
        "metadataType": "LABEL",
        "key": "team",
        "prefix": "k8s."
      }
    }
  ]'
```

10. Troubleshooting

Enrichment Not Appearing

Symptom	Cause	Solution
No enriched attributes	Rules not propagated	Wait 45 minutes after rule creation
Some namespaces missing	namespaceSelector mismatch	Verify DynaKube namespaceSelector
Pods not enriched	metadataEnrichment disabled	Check DynaKube spec

Verify DynaKube Status

```
# Check DynaKube configuration
kubectl -n dynatrace get dynakube -o yaml | grep -A5 metadataEnrichment

# Check for disable annotation
kubectl -n dynatrace get dynakube -o yaml | grep disable-metadata-enrichment
```

Verify Enrichment Files

```
# Check enrichment directory exists
kubectl exec -it &lt;pod-name&gt; -- ls -la /var/lib/dynatrace/enrichment/

# View enrichment content
kubectl exec -it &lt;pod-name&gt; -- cat /var/lib/dynatrace/enrichment/dt_metadata.json
```

Timing Considerations

Action	Propagation Time
New rule created	Up to 45 minutes
Rule modified	Up to 45 minutes
DynaKube deployed first	Immediate (if rules exist)

> **Tip:** If you configure enrichment rules **before** deploying DynaKube, the rules take effect immediately when DynaKube is applied.

Summary

In this notebook, you learned:

- Why settings-based metadata enrichment is the recommended approach
- How to enable enrichment in DynaKube
- Creating enrichment rules in Settings UI
- Using namespace selectors to scope enrichment
- Understanding enrichment file locations and formats
- Querying enriched data with DQL
- Practical use cases: cost allocation, routing, security
- Troubleshooting common issues

References

- Metadata enrichment for K8s telemetry (DT docs)
- Configure enrichment directory (DT docs)
- Settings API — K8s Telemetry Enrichment schema (DT docs)
- K8s security context Grail permissions (DT docs)
- Set up Dynatrace on Kubernetes (DT docs)
- Kubernetes tag setup (DT docs)
- DynaKube parameters (DT docs)
